



Assignment of orthologous genes in unbalanced genomes using cycle packing of adjacency graphs

Gabriel Siqueira¹ · Andre Rodrigues Oliveira² ·
Alexsandro Oliveira Alexandrino¹ · Géraldine Jean³ ·
Guillaume Fertin³ · Zanoni Dias¹

Received: 31 March 2023 / Revised: 10 May 2024 / Accepted: 14 May 2024

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2024

Abstract

The adjacency graph is a structure used to model genomes in several rearrangement distance problems. In particular, most studies use properties of a maximum cycle packing of this graph to develop bounds and algorithms for rearrangement distance problems, such as the reversal distance, the reversal and transposition distance, and the double cut and join distance. When each genome has no repeated genes, there exists only one cycle packing for the graph. However, when each genome may have repeated genes, the problem of finding a maximum cycle packing for the adjacency graph (adjacency graph packing) is NP-hard. In this work, we develop a randomized greedy heuristic and a genetic algorithm heuristic for the adjacency graph packing problem

A preliminary version of this work appeared in the 14th Brazilian Symposium on Bioinformatics (BSB'2021) (Siqueira et al. 2021).

✉ Gabriel Siqueira
gabriel.siqueira@ic.unicamp.br

Andre Rodrigues Oliveira
andro@ic.unicamp.br

Alexsandro Oliveira Alexandrino
alexsandro@ic.unicamp.br

Géraldine Jean
geraldine.jean@univ-nantes.fr

Guillaume Fertin
guillaume.fertin@univ-nantes.fr

Zanoni Dias
zanoni@ic.unicamp.br

¹ Instituto de Computação, Universidade Estadual de Campinas (UNICAMP), Campinas, São Paulo, Brazil

² Faculdade de Computação e Informática, Universidade Presbiteriana Mackenzie, São Paulo, São Paulo, Brazil

³ CNRS, LS2N, UMR 6004, Nantes Université, École Centrale Nantes, Nantes, France

for genomes with repeated genes and unequal gene content. We also propose new algorithms with simple implementation and good practical performance for reversal distance and reversal and transposition distance in genomes without repeated genes, which we combine with the heuristics to find solutions for the problems with repeated genes. We present experimental results and compare the application of these heuristics with the application of the MSOAR framework in rearrangement distance problems. Lastly, we apply our genetic algorithm heuristic to real genomic data to validate its practical use.

Keywords Genome rearrangements · Cycle packing · Transposition · Reversals · Indels

1 Introduction

In the course of evolution, genomes undergo mutations. These mutations can be punctual (i.e., insertion, deletion, or duplication) (Willing et al. 2021; Kahn and Raphael 2008) or large-scale mutations, such as *genome rearrangements* (Oliveira et al. 2024), altering a large stretch of a genome.

In Comparative Genomics, a well-accepted way to infer the distance between two genomes of closely related species is by computing the minimum number of genome rearrangements needed to transform one genome into the other. Several genome rearrangements have been proposed and studied, such as the *reversal* (Bafna and Pevzner 1996), where a segment of genes from the genome is inverted, the *transposition*, which swaps two consecutive segments of genes (Bafna and Pevzner 1998), and the *Double-Cut-and-Join* (DCJ) (Bergeron et al. 2006), that cuts the genome at two positions and creates two new adjacencies by joining the four extremities affected by these cuts.

The first works on genome rearrangements considered genomes sharing the same set of genes and with no repeated genes. These restrictions allow to represent gene order using a permutation, where each element has a ‘+’ or ‘−’ sign indicating gene orientation. The reversal distance and the DCJ distance on signed permutations can be solved in polynomial time (Hannenhalli and Pevzner 1999; Bergeron et al. 2006). If we consider both events of reversal and transposition, the distance problem on signed permutations is NP-hard (Oliveira et al. 2019), and the best known approximation algorithm has factor of 2 (Walter et al. 1998).

Later, mutations capable of altering the content of the genome were also taken into account so that genomes with distinct gene content could be compared. Two of such mutations are the deletion, which removes a block of genes from the genome, and the insertion, which inserts a block of genes in the genome. The term indel refers to a mutation that can be either a deletion or an insertion. The reversal and indel distance, and the DCJ and indel distance on signed permutations can be solved in polynomial time (Willing et al. 2021; Braga et al. 2011). On the other hand, the reversal, transposition, and indel distance on signed permutations is NP-hard and the best known approximation algorithm has factor of 3 (Alexandrino et al. 2021).

Additionally, models considering duplicated or multiple copies of the same gene began to be studied. In this case, it is common to represent genomes as strings, where

labels represent genes, and repeated genes use the same label across the string. Similar to the genome representation using permutations, when gene orientation is known, it can be represented by signs. Although more realistic, most of these problems belong to the NP-hard class of problems (Christie 1998; Radcliffe et al. 2005; Chen et al. 2005; Zhai et al. 2019).

Most results for the rearrangement distance use the adjacency graph (or the equivalent structure called breakpoint graph) (Hannenhalli and Pevzner 1999; Bergeron et al. 2006; Chen et al. 2005) to model the two compared genomes. This structure is defined in Sect. 2. More precisely, bounds and algorithms rely on cycle packings of this graph. When there are no repeated genes and the genomes share the same set of genes, only one cycle packing exists, and it is trivial to find it (Hannenhalli and Pevzner 1999).

Any cycle packing of an adjacency graph induces a one-to-one assignment between genes of the same label. The size of a cycle packing, meaning the number of cycles in it, is inversely related to the value of upper and lower bounds for distances involving reversal, DCJ, and transposition. In this way, we are interested in finding maximum size cycle packings of adjacency graphs. However, when there are repeated genes, the problem of finding a maximum size cycle packing of an adjacency graph, called *Adjacency Graph Packing* problem, is NP-hard (Shao et al. 2015). A similar work on cycle packing was recently presented by Pinheiro et al. (2020), but they assumed no repeated genes, and they did not consider gene orientation.

Other studies applied the cycle packing approach to genomes with unequal gene content. In particular, Bohnenkämper et al. (2021) proposed an Integer Linear Programming (ILP) model to solve a variation of the adjacency graph packing problem. They use this model to compute the DCJ and indel distance between two genomes.

In this work, we propose another adaptation of the adjacency graph to deal with genomes with unequal gene content and create heuristics to find cycle packings of an adjacency graph with as many cycles as possible when considering repeated genes and gene orientation. We present two heuristics, one using a randomized greedy strategy, and one using a Genetic Algorithm (Mitchell 2008). We test our heuristics on real and simulated genomes and compare them against the MSOAR framework (Fu et al. 2007) and the ILP of Bohnenkämper et al. (2021).

This work is divided as follows. Section 2 presents all the definitions used throughout the manuscript. Section 3 proposes a randomized greedy heuristic for the adjacency graph packing problem. Section 4 defines a heuristic based on the Genetic Algorithm technique. Section 5 shows the application of the heuristics in rearrangement distances. Section 6 presents the experiments with the proposed heuristics and the algorithms from the literature. Section 7 concludes the paper.

2 Definitions

A genome $\mathcal{G} = (g_1 \ g_2 \ \dots \ g_n)$ is treated as a sequence of n genes labeled in order as $g_1, g_2, \dots, g_n$. We represent $\mathcal{G}$ by a *signed string* S , where each character S_i (i.e., the character at position i) corresponds to a gene g_i and it has a '+' or '-' sign representing the gene orientation.

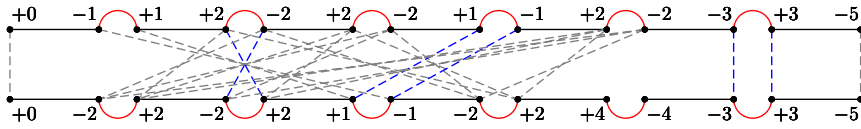


Fig. 1 The adjacency graph of two strings $S = (+0 +1 -2 -2 -1 -2 +3 +5)$ and $P = (+0 +2 +2 -1 +2 -4 +3 +5)$. Black edges are represented by continuous lines, red edges are represented by arcs, and gray edges are represented by dashed lines. Some twin gray edges are marked in blue

The *size* of S is denoted by $|S|$. The *alphabet* of a string S , denoted by Σ_S , is the set of all characters of S ignoring their signs. In this work, we use numbers as the characters of the strings that represent the genomes.

A character $\alpha \in \Sigma_S$ appears in S one or more times, and each one is called an *occurrence* of α in S . The number of occurrences of α in S is denoted by $occ(\alpha, S)$.

Given a string S , a character α is called a *singleton*, if $occ(\alpha, S) = 1$, and it is called a *replica* otherwise.

Example 1 A string S and some information of this string. The character 2 is a singleton, while all other characters are replicas.

$$S = (+1 +2 -1 +4 -3 +1 -4 +3)$$

$$\Sigma_S = \{1, 2, 3, 4\}, |S| = 8, S_1 = +1, S_7 = -4$$

$$occ(1, S) = 3, occ(2, S) = 1, occ(3, S) = occ(4, S) = 2$$

Let us assume that every string being compared has size at most M . We *extend* a string S by adding the elements $S_0 = +0$ and $S_{|S|+1} = +(M+1)$. Henceforward, we assume that strings are in their extended form unless stated otherwise.

Given a string S , we define the *partial graph* $G_S = (V_S, E_S)$, with vertex set V_S and edge set $E_S = E_S^b \cup E_S^r$. For every $0 \leq i \leq |S|$, there exist two vertices in V_S of labels $+S_i$ and $-S_{i+1}$ connected by an undirected edge $e_i^b \in E_S^b$, which is classified as a *black edge*. Note that each character S_i in S , except for $+0$ and $+(M+1)$, corresponds to two vertices $+S_i$ and $-S_i$. We say that such vertices are *twins*, and we connect them with an undirected edge e_i^r , which is classified as a *red edge*. The character $+0$ uniquely corresponds to the vertex $+0$ and the character $+(M+1)$ uniquely corresponds to the vertex $-(M+1)$. Consequently, they do not correspond to twin vertices.

Given two strings S and P , the *adjacency graph* $G(S, P) = (V, E)$ includes the disjoint union of the partial graphs G_S and G_P , and the set of gray edges E_g . For every pair $u \in V_S$ and $v \in V_P$, there exists a gray edge $\{u, v\}$ in E_g if these vertices have the same label. Two gray edges $\{u, v\}$ and $\{t, s\}$ are called *twins* if u and t are twins, and v and s are also twins. Figure 1 shows an example of the adjacency graph for two strings.

An *alternating cycle* of an adjacency graph $G(S, P)$ is a cycle composed of a sequence of edges $(e_0, e_1, e_2, \dots, e_t)$, such that e_i is black if i is even and e_i is gray

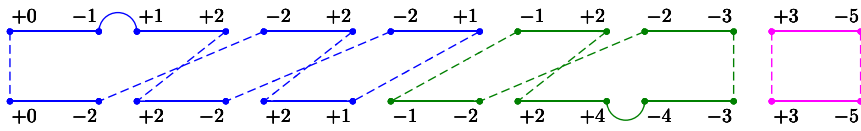


Fig. 2 A cycle packing for the adjacency graph of two strings $S = (+0 +1 -2 -2 -1 -2 +3 +5)$ and $P = (+0 +2 +2 -1 +2 -4 +3 +5)$. The cycle packing has three different alternating cycles

or red if i is odd. An *alternating cycle packing* (or just *cycle packing*) of the graph $G(S, P)$ is a set of disjoint alternating cycles, such that:

- Each vertex belongs to exactly one alternating cycle.
- A gray edge $\{u, v\}$ belongs to a cycle if and only if it does not have a twin, or its twin edge is also in some cycle.
- The amount of red edges from G_S connecting vertices $+\alpha$ and $-\alpha$ from some cycle is equal to $\max(0, \text{occ}(\alpha, S) - \text{occ}(\alpha, P))$.
- The amount of red edges from G_P connecting vertices $+\alpha$ and $-\alpha$ from some cycle is equal to $\max(0, \text{occ}(\alpha, P) - \text{occ}(\alpha, S))$.

Figure 2 shows a cycle packing of the adjacency graph from Fig. 1.

Given two strings S and P , the set of *common genes* of S and P is the set $\Sigma_{SP} = \Sigma_S \cup \Sigma_P$. Given an alternating cycle packing of $G(S, P)$, the gray edges encode a one-to-one mapping between occurrences of genes from Σ_{SP} . More precisely, for every $\alpha \in \Sigma_{SP}$, there are $\min(\text{occ}(\alpha, S), \text{occ}(\alpha, P))$ correspondences between an occurrence of α in S and an occurrence of α in P . Two genes are correspondent if the vertices originated from them are connected by gray edges. Note that the second condition of cycle packing ensures that we do not have vertices originated from a gene of one genome connected with vertices from distinct genes of the other genome.

In the adjacency graph packing problem, given an adjacency graph $G(S, P)$ and a gain function g that associates an integer with each alternating cycle packing of G , the goal is to find an alternating cycle packing $\mathcal{H}^*$ of G such that the value of g is maximized. We will present our algorithm with a generic gain function in Sects. 3 and 4. In Sect. 5, we describe a particular gain function for the rearrangement problems.

3 Random packings

Our first approach to solve the adjacency graph packing problem is the Random Packings (RP) heuristic. This heuristic randomly generates a set of M cycle packings and selects the one with the maximum gain. Besides the strings S and P , the heuristic receives a parameter r indicating the number of cycle packings to be generated. The BFS_PACK procedure is responsible for generating each random cycle packing and is explained next.

A *partial alternating cycle packing* (or just *partial cycle packing*) $\mathcal{H}$ of the graph $G(S, P)$ is a set of disjoint alternating cycles, such that:

1. Each vertex belongs to at most one alternating cycle.

2. A gray edge $\{u, v\}$ belongs to a cycle if and only if it does not have a twin, its twin edge is also in some cycle, or it is possible to include a cycle in $\mathcal{H}$ containing its twin edge such that $\mathcal{H}$ remains a partial alternating cycle packing.
3. The amount of red edges from G_S connecting vertices $+\alpha$ and $-\alpha$ from some cycle is at most $\max(0, \text{occ}(\alpha, S) - \text{occ}(\alpha, P))$.
4. The amount of red edges from G_P connecting vertices $+\alpha$ and $-\alpha$ from some cycle is at most $\max(0, \text{occ}(\alpha, P) - \text{occ}(\alpha, S))$.

In other words, a partial alternating cycle packing is an alternating cycle packing with more flexible restrictions allowing it to not cover all the vertices.

The `BFS_PACK` procedure initiates with a possibly empty partial cycle packing $\mathcal{H}$ and inserts cycles in it until enough cycles are inserted to cover all vertices, in this way, $\mathcal{H}$ becomes a cycle packing. Additionally, every insertion must ensure that $\mathcal{H}$ remains a partial cycle packing.

Each cycle is selected using a breadth-first search (BFS) in the adjacency graph. At each iteration, the search for a new alternating cycle starts in a randomly chosen vertex v , such that v is not yet covered by any cycle of $\mathcal{H}$. The search direction is guided by following the black edge incident to v . At each iteration, the procedure follows a black edge and chooses a gray or red edge for the subsequent step, continuing this sequence until it returns to v . The selection of gray or red edges must adhere to the following constraints to ensure that the new cycle does not violate the conditions of a partial cycle packing:

- The search never uses an edge incident to a vertex belonging to a cycle already in $\mathcal{H}$ (to ensure that the first condition is met).
- If an edge is in a cycle and has a twin edge, the twin edge must also be in a cycle (to ensure that the second condition is met).
- The search does not follow a red edge from G_S connecting vertices $+\alpha$ and $-\alpha$ if there are $\max(0, \text{occ}(\alpha, S) - \text{occ}(\alpha, P))$ of such red edges already in cycles of the packing (to ensure that the third condition is met).
- The search does not follow a red edge from G_P connecting vertices $+\alpha$ and $-\alpha$ if there are $\max(0, \text{occ}(\alpha, P) - \text{occ}(\alpha, S))$ of such red edges already in cycles of the packing (to ensure that the fourth condition is met).

The search stops when the first cycle is found. At that point, the new cycle is added to $\mathcal{H}$ and, if $\mathcal{H}$ is not yet a cycle packing, a new search starts. Note that all vertices not covered by a cycle in $\mathcal{H}$ have an available gray or red edge and an available black edge, therefore, while the search does not find a cycle, it is possible to follow a black, gray, or red edge to another vertex. Eventually, the search finds an alternating cycle of G not yet in $\mathcal{H}$. To uphold the second restriction, we can eliminate all other red and gray edges incident to vertices from the twin of each selected edge.

This application of BFS introduces a greedy aspect to the procedure, as it identifies one of the smallest cycles starting from the randomly chosen vertex.

Algorithm 1 presents a pseudocode of the Random Packings heuristic, where lines 4 to 7 show the `BFS_PACK` procedure. It is worth noting that the `BFS_PACK` procedure can be applied to any partial cycle packing (not necessarily empty) in order to obtain a cycle packing. Figure 3 shows an example of the `BFS_PACK` procedure.

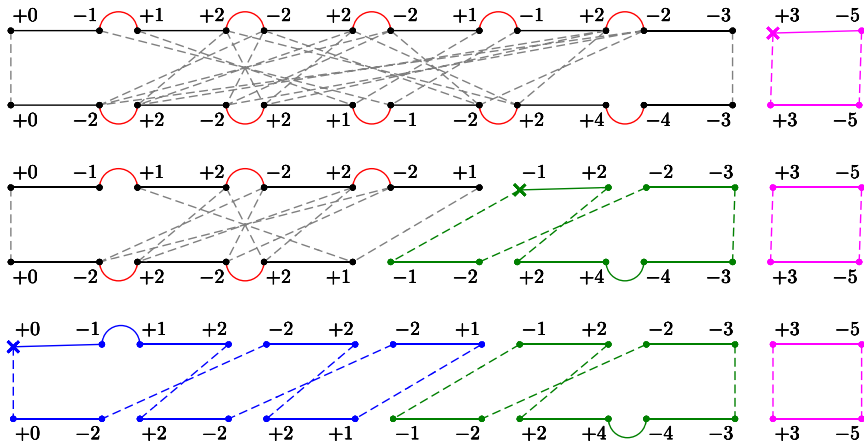


Fig. 3 Example of the `BFS_PACK` procedure applied to the adjacency graph of two strings $S = (+0 +1 -2 -2 -1 -2 +3 +5)$ and $P = (+0 +2 +2 -1 +2 -4 +3 +5)$. The cycles obtained on each breadth-first search are marked with an $\times$ indicating the starting vertex of the search. Edges that can not be used in future searches are omitted from the figure

Algorithm 1 Random Packings

Require: strings S and P , number of cycle packings r , and gain function g

Ensure: returns a cycle packing for $G(S, P)$

```

1:  $\mathbf{M} \leftarrow \emptyset$ 
2: while  $|\mathbf{M}| < r$  do
3:    $\mathcal{H} \leftarrow \emptyset$ 
4:   while  $\mathcal{H}$  does not cover all vertices of  $G(S, P)$  do
5:      $v \leftarrow$  vertex of  $G(S, P)$  not belonging to any cycle of  $\mathbf{M}$ 
6:      $C \leftarrow$  cycle resulting from a breadth-first search in  $G(S, P)$ , starting with  $v$  and following the
       necessary restrictions
7:      $\mathcal{H} \leftarrow \mathcal{H} \cup C$ 
8:   end while
9:    $\mathbf{M} \leftarrow \mathbf{M} \cup \{\mathcal{H}\}$ 
10: end while
11: return Cycle packing belonging to  $\mathbf{M}$  with maximum value of  $g$ 
    
```

4 Genetic algorithm

In this section, we present our algorithm based on the Genetic Algorithm (GA) approach (Mitchell 2008). This strategy was inspired by the process of evolution, where given an initial population new generations arise from crossover and mutations of individuals from the previous generation. Furthermore, individuals more adapted to survive have a higher chance of participating in the crossovers. The idea behind genetic algorithms is to represent each solution as an individual and the value of that solution as the fitness of that individual. In the adjacency graph packing problem, each individual is a cycle packing and their fitness is the gain of the cycle packing.

Our genetic algorithm has three parameters: the total number of individuals to be generated r , the size of the population p , and the mutation rate m . The algorithm starts

with an initial population $\mathbf{P}_0$ of individuals generated by the Random Packings heuristic and, until we reach a total of r individuals, a new population $\mathbf{P}_{i+1}$ of individuals is produced using the individuals from the previous population $\mathbf{P}_i$. The algorithm takes the following steps in order to generate a new population:

1. **Selection:** At this step, the algorithm produces a set $\mathbf{S}$ with p pairs of individuals to take part in the crossovers. Each individual of a pair is selected by tournament, in which two individuals are randomly chosen and the one with the highest fitness is selected. A new pair is added to $\mathbf{S}$, where the two individuals are selected by two tournaments, until $\mathbf{S}$ has size p . The random selection of individuals for the tournament allows repetitions, so that an individual may belong to multiple pairs of $\mathbf{S}$.
2. **Crossover:** For each pair of $\mathbf{S}$, the algorithm applies the crossover operation to generate a new individual. Given two cycle packings $\mathcal{H}$ and $\mathcal{H}'$ of an adjacency graph $G(S, P)$, a *crossover* of $\mathcal{H}$ and $\mathcal{H}'$ is a new cycle packing created by the following procedure. Let L and L' be two randomly ordered lists of the cycles from $\mathcal{H}$ and $\mathcal{H}'$, respectively. Starting with an empty set $\mathcal{H}''$, while $\mathcal{H}''$ is not a cycle packing and there are cycles in L or L' , remove a cycle from L or L' (with a 50% probability to remove from each list, or 100% probability to remove from one list if the other is empty), and add it to $\mathcal{H}''$ if after the addition $\mathcal{H}''$ remains a partial cycle packing. If both lists L and L' are empty and $\mathcal{H}''$ is not yet a cycle packing, use the BFS_PACK procedure to complete the packing.
3. **Mutation:** After the new individuals have been generated, a mutation is applied to each individual in order to increase the diversity of the population. In the mutation of a cycle pack $\mathcal{H}''$, we remove each cycle of $\mathcal{H}''$ with probability m and create a new packing using the BFS_PACK procedure starting with the remaining cycles of $\mathcal{H}''$.
4. **Elitism:** As we are going to replace the old population with the new one, we cannot guarantee the quality of the new population. So, to ensure that at least the best individual is kept, we replace the individual with the lowest fitness from the new population with the individual with the highest fitness from the old population, if the old individual has a higher fitness than the new one.

Algorithm 2 presents a pseudocode for the Genetic Algorithm heuristic. Figure 4 exemplifies the generation of a new cycle packing after the crossover of two previous ones followed by a mutation.

5 Applications with rearrangement distance

This section explores the application of the adjacency graph packing problem to calculate rearrangement distances involving the events of reversal, transposition, and indel.

A *reversal* is a large-scale mutation that inverts a subsequence of genes (and, consequently, their orientations). Formally, a reversal $\rho(i, j)$, with $1 \leq i \leq j \leq n$, when applied to a given signed string $S = (S_0 \dots S_{i-1} S_i \dots S_j S_{j+1} \dots S_{n+1})$ generates a new string $S' = S \cdot \rho(i, j) = (S_0 \dots S_{i-1} \underline{-S_j \dots -S_i} S_{j+1} \dots S_{n+1})$.

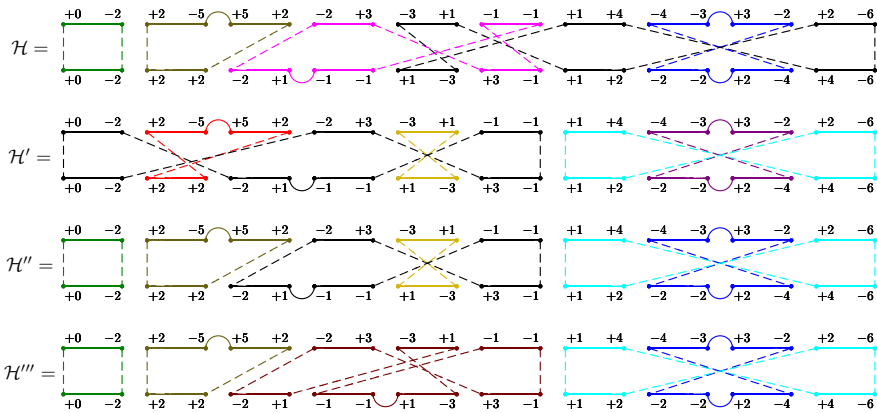


Fig. 4 The two cycle packings $\mathcal{H}$ and $\mathcal{H}'$ are combined by crossover resulting in the cycle packing $\mathcal{H}''$, where three cycles are copied from $\mathcal{H}$, two cycles are copied from $\mathcal{H}'$, and one cycle is inserted by the BFS_PACK procedure. Afterwards, the cycle packing $\mathcal{H}''$ suffers a mutation resulting in the cycle packing $\mathcal{H}'''$, where two cycles are removed and a new cycle is inserted by the BFS_PACK procedure

Algorithm 2 Genetic Algorithm

Require: strings S and P , number of cycle packings r , size of the population p , mutation rate m , and gain function g

Ensure: returns a cycle packing for $G(S, P)$

```

1:  $\mathbf{P}_0 \leftarrow p$  cycle packings generated by the Random Packings heuristic
2:  $\mathbf{M} \leftarrow \mathbf{P}_0$ 
3:  $i \leftarrow 0$ 
4: while  $|\mathbf{M}| < r$  do
5:    $\mathbf{S} \leftarrow$  Sequence of  $p$  selected pairs of individuals from  $\mathbf{P}_i$ 
6:    $\mathbf{P}'_{i+1} \leftarrow$  Set of  $p$  new individuals created by crossovers applied in  $\mathbf{S}$ 
7:    $\mathbf{P}''_{i+1} \leftarrow$  Set of  $p$  individuals obtained by mutation applied in  $\mathbf{P}'_{i+1}$ 
8:   if the best individual from the set  $P_i$  is better than the worst individual from the set  $P''_{i+1}$  then
9:      $\mathbf{P}_{i+1} \leftarrow$  Set  $\mathbf{P}''_{i+1}$  with its worst individual replaced with the best individual from  $P_i$ 
10:   else
11:      $\mathbf{P}_{i+1} \leftarrow \mathbf{P}'_{i+1}$ 
12:   end if
13:    $\mathbf{M} \leftarrow \mathbf{M} \cup \{\mathbf{P}_{i+1}\}$ 
14:    $i \leftarrow i + 1$ 
15: end while
16: return Cycle packing of  $\mathbf{M}$  with maximum value of  $g$ 
    
```

A *transposition* is a large-scale mutation that swaps two consecutive subsequences of genes (without affecting their orientations). Formally, a transposition $\tau(i, j, j)$, with $1 \leq i < j < k \leq n + 1$, when applied to a given signed string $S = (S_0 \dots S_{i-1} S_i \dots S_{j-1} S_j \dots S_{k-1} S_k \dots S_{n+1})$ generates a new string $S' = S \cdot \tau(i, j, k) = (S_0 \dots S_{i-1} S_j \dots S_{k-1} S_i \dots S_{j-1} S_k \dots S_{n+1})$.

An *insertion* is a mutation that inserts a subsequence of genes. Formally, an insertion $\phi(i, Q)$, such that $0 \leq i \leq n$ and Q is a signed string without extension, when applied to a given signed string $S = (S_0 \dots S_i S_{i+1} \dots S_{n+1})$ generates a new string $S' = S \cdot \phi(i, Q) = (S_0 \dots S_i Q_1 \dots Q_{|Q|} S_{i+1} \dots S_{n+1})$, such that $|S'| = |S| + |Q|$.

A *deletion* is a mutation that removes a subsequence of genes. Formally, a deletion $\psi(i, j)$, with $1 \leq i \leq j \leq n$, when applied to a given signed string $S = (S_0 \dots S_{i-1} S_i \dots S_j S_{j+1} \dots S_{n+1})$ generates a new string $S' = S \cdot \psi(i, j) = (S_0 \dots S_{i-1} S_{j+1} \dots S_{n+1})$, such that $|S'| = |S| - (j - i + 1)$.

The *Reversal and Indel Distance* problem (RID) seeks the minimum number of reversals, insertions, and deletions required to transform one genome into the other, which is defined as the *reversal and indel distance*. If the strings only have common characters, then we do not need indels, and we have the *Reversal Distance* problem (RD), that seeks the minimum number of reversals required to transform one genome into the other, defined as the *reversal distance*.

The *Reversal, Transposition, and Indel Distance* problem (RTID) seeks the minimum number of reversals, transpositions, insertions, and deletions required to transform one genome into the other, which is defined as the *reversal, transposition and indel distance*. If the strings only have common characters, then we do not need indels, and we have the *reversal and transposition distance* problem (RTD), which seeks the minimum number of reversals and transpositions required to transform one genome into the other, which is defined as the *reversal and transposition distance*.

As stated previously, an Alternating Cycle Packing induces a one-to-one mapping between common genes of two genomes. In the distance problems, given such mapping, we can assume that every gene without correspondent has to be inserted or deleted, and the remaining genes must be rearranged (with reversals or transpositions) according to the correspondence to transform one genome into the other. That allows us to use solutions for the distances problems, restricted to the case where every gene has at most one occurrence in each genome. Such solutions will give us a sequence of rearrangement operations (among reversal, transpositions, insertions, or deletions, depending on the problem) capable of transforming one genome into the other. Note that the size of any such sequence is an upper bound for the actual distance between the genomes.

Every Alternating Cycle Packing of the adjacency graph will produce one sequence of operations. So we are interested in a gain function for the heuristics to select an Alternating Cycle Packing that produces a short sequence of operations. We use the following definitions to define such gain function.

A run in an Alternating Cycle Packing is a maximal path that only contains red edges of one genome. We denote the number of runs in a cycle C of an Alternating Cycle Packing by $\Lambda(C)$. The indel potential $\lambda(C)$ of a cycle C is a value related to the number of indels necessary to remove all runs of that cycle (Braga et al. 2011) and is defined by:

$$\lambda(C) = \begin{cases} \left\lceil \frac{\Lambda(C)+1}{2} \right\rceil, & \text{if } \Lambda(C) > 0 \\ 0, & \text{otherwise} \end{cases}$$

The gain function we are going to use in our heuristics to evaluate an Alternating Cycle Packing $\mathcal{H}$ is defined by $g(\mathcal{H}) = |\mathcal{H}| - \sum_{C \in \mathcal{H}} \lambda(C)$. We choose this function because it is related to the algorithm we are using to calculate the results for the RID and RTID problems. Such algorithm is described in the remaining of this section.

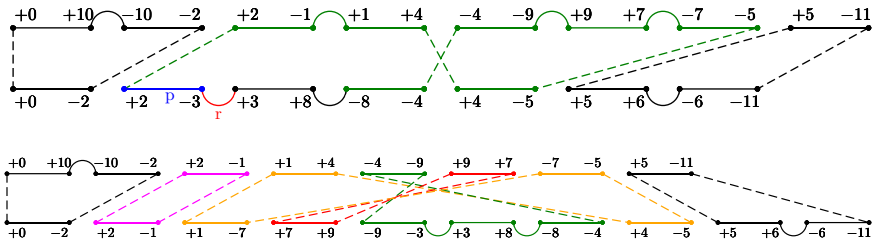


Fig. 5 Effect of the $REM_RUN(R, S, P)$ procedure in the cycle graph $G(S, P)$ from the strings $S = (+0 -10 +2 +1 -4 +9 -7 +5 +11)$ and $P = (+0 +2 +3 -8 +4 +5 -6 +11)$. The image on the top shows the original graph and the run $R = [-3, +2, +2, -1, +1, +4, +4, -5, -5, -7, +7, +9, -9, -4, -4, -8]$, which starts in the black edge p (colored in blue) after the red edge r , so it only includes red edges from S . The orientation o is right to left. In this example, the sequence Q is $[+1, +7, -9]$, note that $+1$ is visited in the orientation left to right (distinct from o), and -7 and $+9$ are visited in the orientation o . The image on the bottom shows the resulting graph, with C separated into 4 cycles. The genome P after the insertion is $(+0 +2 +1 +7 -9 +3 -8 +4 +5 -6 +11)$

5.1 Algorithms for rearrangement distances without replicas

The exact polynomial time algorithm for reversal and indel distance without replicas (Willing et al. 2021) has not yet been implemented. In this section, we propose a 1.5-approximation algorithm that is easier to implement and obtains results close to the optimum in practice. The same approach can be applied for other rearrangement distances that include indels. In particular, we use a similar algorithm to calculate the reversal, transposition, and indel distance in our practical experiments.

The main idea of the algorithm is to remove all characters with occurrences in only one of the strings, and apply an algorithm for the case where both strings have the same set of characters.

In the remaining of this section, we assume that we are comparing two strings without replicas S and P . As the strings S and P do not have replicas, the adjacency graph $G(S, P)$ has a unique alternating cycle packing $\mathcal{H}$. We are going to apply the following procedure (which we call $REM_RUN(R, S, P)$) to remove a run R from the packing $\mathcal{H}$:

- Without loss of generality, assume that the red edges from the run correspond to characters of S .
- If the run is in a cycle that has more than one run (see Fig. 5 for an illustration):
 - Let r be one of the red edges, correspondent to a character of P , connected to one of the extremities of the run. Note that a red edge exists because the run is a maximal path and the cycle has more than one run.
 - Starting from r , follow the run and construct a sequence Q of genes corresponding to each red edge from the run. Let b be a black edge of the run that is first visited while constructing Q , and let o be the orientation in which b is visited (left to right or right to left). Associate a sign with each character from Q , such that a character receives a positive sign if its red edge is visited in the orientation o and the original character has a negative sign or if its red edge is

- visited in the other orientation and the original character has a positive sign. Otherwise, the character receives a negative sign.
- If b is visited from left to right, invert the order of the elements in the sequence Q , otherwise leave Q unchanged. Next, insert the sequence Q on the right of the character correspondent to the left vertex of b .
 - If the run is in a cycle that has a single run:
 - Let b be a black edge from P in the cycle containing the run.
 - Starting from b , follow the run, in any direction, and construct a sequence Q of genes corresponding to each red edge from the run. Let o be the orientation in which b is visited when constructing Q (left to right or right to left). Associate a sign with each character from Q , such that a character receives a positive sign if its red edge is visited in the orientation o and the original character has a negative sign or if its red edge is visited in the other orientation and the original character has a positive sign. Otherwise, the character receives a negative sign.
 - If b is visited from right to left, invert the order of the elements in the sequence Q , otherwise leave Q unchanged. Next, insert the sequence Q on the right of the character correspondent to the left vertex of b .

Lemma 1 *Given two strings S and P , let R be a run from the cycle packing $\mathcal{H}$ of $G(S, P)$. The $REM_RUN(R, S, P)$ procedure decreases by one the value of $|\Sigma_{SP}| - |\mathcal{H}| + \sum_{C \in \mathcal{H}} \lambda(C)$.*

Proof Without loss of generality, assume that the $REM_RUN(R, S, P)$ inserts characters in P (i.e., the red edges from the run R corresponds to characters of S). In that case, the procedure inserts a sequence $Q = (Q_1, \dots, Q_{|Q|})$ of characters, equal to characters from S , in P . Consequently, the value of $|\Sigma_{SP}|$ increases by $|Q|$.

Let C be the cycle containing the run R . Note that, every inserted pair (Q_i, Q_{i+1}) , for $1 \leq i \leq |Q| - 1$, will split a subgraph of C into a new cycle, because the red edges of the correspondent characters in S will be removed and their vertices will be connected by gray edges with the inserted characters (an example is shown in Fig. 5). For a similar reason, the remaining vertices of the cycle C will be separated into two cycles created by the gray edges inserted between Q_1 and its correspondent in S and $Q_{|Q|}$ and its correspondent in S . That causes the insertion of $|Q|$ new cycles in $\mathcal{H}$. Consequently, the value of $|\Sigma_{SP}| - |\mathcal{H}|$ does not change.

Now, we look at the number of runs in $\mathcal{H}$. First, note that the number of runs in a cycle is either one or an even number (Alexandrino et al. 2022). If C has a single run, all the cycles created by splitting C will have an indel potential of zero, reducing the value of $\sum_{C \in \mathcal{H}} \lambda(C)$ by one. If C has two runs, then the procedure removes one run from C and leaves a single run in the created cycles, reducing the value of $\sum_{C \in \mathcal{H}} \lambda(C)$ by one. If C has four or more runs, then the procedure removes one run from C and places all remaining runs from C in a single cycle, such that two of those runs are merged into one. Consequently, the value of $\sum_{C \in \mathcal{H}} \lambda(C)$ is also reduced by one. $\square$

Let $R_ALG(S, P)$ be an algorithm that produces a sequence of reversals (or reversals and transpositions) capable of transforming S into P if they have only common characters. Consider Algorithm 3, that uses R_ALG and the procedure REM_RUN .

Algorithm 3 Distance Algorithm**Require:** strings S and P **Ensure:** returns a sequence of reversals and indels (or reversals, transpositions, and indels) capable of transforming S into P

```

1:  $\mathcal{H} \leftarrow$  cycle packing of  $G(S, P)$ 
2:  $L' \leftarrow$  empty sequence
3:  $L'' \leftarrow$  empty sequence
4: while there is a run  $R$  in  $\mathcal{H}$  do
5:    $S, P \leftarrow$  strings from the  $REM\_RUN(R, S, P)$  procedure
6:   if the  $REM\_RUN(R, S, P)$  procedure inserts characters in  $S$  then
7:     Append to  $L'$  the insertion used in the  $REM\_RUN(R, S, P)$  procedure
8:   else
9:     Append to  $L''$  the insertion used in the  $REM\_RUN(R, S, P)$  procedure
10:  end if
11:  $\mathcal{H} \leftarrow$  cycle packing of  $G(S, P)$ 
12: end while
13:  $L''' \leftarrow$  Sequence from  $R\_ALG(S, P)$ 
14: Reverse the sequence  $L''$  and replace each insertion with a deletion of the inserted characters
15:  $L \leftarrow$  Concatenation of the sequences  $L', L'''$  and  $L''$ , in that order
16: return  $L$ 

```

Using an exact polynomial time algorithm (Hannenhalli and Pevzner 1999) to find the reversal distance between two strings without replicas and with only common genes, we can prove an approximation factor of 1.5 for Algorithm 3 applied to the reversal and indel distance.

Theorem 1 *If Algorithm 3 uses as $ALG(S, P)$ an exact algorithm to find a sequence of reversals of minimal size capable of transforming S into P , then it is a 1.5-approximation for the RID problem between two strings without replicas.*

Proof First, we argue that Algorithm 3 finds a sequence of reversals and indels capable of transforming S into P . The final sequence L has a sequence of deletions L' that transforms S into a string S' , a sequence of reversals L''' that transforms S' into a string P' , and a sequence of deletions L'' that transforms P' into P . The sequences L' and L'' exist by the definition of the REM_RUN procedure. The strings S' and P' correspond to the strings S and P after all applications of the REM_RUN procedure. Consequently, S and P only have common genes and the sequence L''' exists.

Now, we prove the approximation ratio. We know that $lb(S, P) = |\Sigma_{SP}| - 1 - |\mathcal{H}| + \sum_{C \in \mathcal{H}} \lambda(C)$ is a lower bound for the reversal distance (Willing et al. 2021). From Lemma 1, each application of the REM_RUN procedure will use one operation of insertion in one of the strings to reduce the value of $lb(S, P)$ by one. Note that, each reversal applied in P will be converted into a deletion applied in S , without changing the total number of operations.

After removing the runs, we are using an exact algorithm for the reversal distance as $ALG(S, P)$ to produce a sequence of reversals capable of transforming S into P . Brito et al. (2021, Theorem 7) showed that the reversal distance $d(S, P)$ between strings without replicas and with only common genes is such that $d(S, P) \leq 1.5(|\Sigma_{SP}| - 1 - |\mathcal{H}|)$. As $ALG(S, P)$ is an exact algorithm and $\sum_{C \in \mathcal{H}} \lambda(C) = 0$ at this point, $ALG(S, P)$ can transform S into P with at most $1.5lb(S, P)$ reversals.

Combining the two previous arguments, we know that the sequence L has size of at most $1.5lb(S, P)$ and the theorem follows. $\square$

6 Experimental results

To test the proposed heuristics, we created a dataset comprising simulated genomes and used a dataset of real genomes. We also tested the MSOAR framework developed by Fu et al. (2007), which offers an alternative method to produce an assignment between genes with the same label, and the ILP developed by Bohnenkämper et al. (2021). For the tests, we produced assignments between the genes with the heuristics, the MSOAR framework, and the ILP, ensuring that after the assignment every label appears in only one gene. Afterwards, we applied algorithms to the resulting genomes to calculate the rearrangement distance.

The MSOAR framework comprises four steps: (i) The application of suboptimal rules to reduce the number of replicas without a significant impact on the resulting distance; (ii) the simplification of the input strings with the removal of more copies of genes; (iii) the construction of a cycle packing using a heuristic similar to the BFS_PACK procedure, but without the red edges; and (iv) the removal of noise genes. During the simplification in step (ii) the MSOAR framework uses a heuristic for the Signed Common Minimum Partition Problem which transforms blocks of genes (characters) in a single character while removing some characters to ensure that the final strings have the same occurrence of each label. We are going to apply step (i) before the application of our heuristics as well, because it has a small impact on the distance and it reduces the number of cycle packings that we need to explore. Besides, step (iv) uses a heuristic to remove some genes that are not in a true biological correspondence, but as we are interested in an assignment with all genes, we disabled that step. Consequently, our comparison is between our heuristics and steps (ii) and (iii) of the MSOAR framework.

6.1 Experiments with simulated genomes

Our dataset of simulated genomes consists of 2500 pairs of strings of size 200. We opted to simulate genomes that are considerably smaller than real-sized genomes, so that we could perform tests with a large number of replicas and occurrences relative to the genome size, the main factor influencing algorithm performance. Our dataset is divided into 25 sets of 100 pairs.

Each set uses an alphabet Σ of a specific size ($|\Sigma|$), such that sizes of alphabets range from 10 to 50 in intervals of 10, and a number of operations O , which also ranges from 10 to 50 in intervals of 10. Each pair (S, T) of signed strings representing the source (S) and target (T) strings was generated as follows. We first constructed the source string S of size 200 such that, at each position $i \in [1..200]$, we chose (with replacement and using a uniform distribution) a character from Σ . Next, each element from S randomly received a sign ('+' or '-'). After that, the target string T was generated by applying O operations of reversal, or reversal and transpositions,

Table 1 The average number of replicas (REP) and average maximum occurrence of a replica (OCC) by alphabet size ($|\Sigma|$)

$ \Sigma $	10	20	30	40	50
REP	10.00	19.99	29.99	38.37	45.28
OCC	26.68	16.15	12.26	10.27	8.99

in S followed by $\lfloor \frac{O}{4} \rfloor$ operations of deletion (deleting a single gene each) and $\lfloor \frac{O}{4} \rfloor$ operations of insertion (inserting a single gene each).

To show the impact of alphabet size on the number of replicas and occurrences of each replica, Table 1 shows the average number of replicas (number of characters with two or more copies) and the average maximum occurrence of a replica in the genomes created with each alphabet size.

The 2500 pairs are separated into two groups based on the operations used to produce the target strings. The first group was used in tests with the RID problem. In that group, the target string was produced with O operations of reversal plus the operations of insertion and deletion. The second group was used in tests with the RTID problem. In that group, the target string was produced with $\lfloor \frac{O}{2} \rfloor$ operations of reversal and $\lceil \frac{O}{2} \rceil$ operations of transposition (the order of the operations was randomly chosen with a uniform distribution), plus the operations of insertion and deletion.

In our tests, for each pair of source and target genomes, we used the heuristics, the MSOAR framework, and the ILP to map the common replicas, and applied the distance algorithms from Sect. 5.1. The reversal and indel distance algorithm was used in the first group, and the reversal, transposition, and indel distance algorithm was used in the second group. To ensure that not using an exact algorithm for the reversal and indel distance without replicas would not considerably affect our tests, we compared the generated distances with the lower bound. The comparison shows that no distance exceeded the lower bound by more than 3%. For the reversal, transposition, and indel distance without replicas, the distances are at most twice the lower bound. We recall that this second problem is NP-hard and even without indels the best known approximation factor for it is 2 (Walter et al. 1998).

The simulated dataset and the C++ implementation of the Random Packing heuristic, the Genetic Algorithm heuristic, and the new algorithms for the distances are available in a public repository.¹ For MSOAR we used the C++ implementation from Shi.² For the ILP we used the implementation provided by the authors,³ and limited the running time for each instance to 2 minutes. The experiments were conducted on a computer equipped with a 2.3GHz Intel® Xeon® CPU E5-2650 v3, with 40 cores and 32 GB of RAM, running Ubuntu 18.04.2.

One relevant aspect of the heuristics is that they may provide different results in each execution due to their randomized aspect. To assess this variation, we performed multiple executions of the GA heuristic in a subset of instances from our database. We chose to use instances produced with an alphabet of size 10, because they have a higher number of replicas, leading to a greater variety of cycle packings. For each

¹ <https://github.com/compbiogroup/Heuristics-for-Cycle-Packing-of-Adjacency-Graphs>.

² <https://github.com/fishinucr/msoar2.0>.

³ <https://gitlab.ub.uni-bielefeld.de/gi/dingiiofficial>.

Table 2 The average (Avg.) and the average of the standard deviations (Std.) of the resulting reversal distances, considering 100 executions of the GA heuristic for the 10 instances selected for each value of O

O	10	20	30	40	50
Avg.	29.19	58.22	80.87	93.08	106.80
Std.	0.14	1.40	1.82	1.85	1.85

Table 3 Average reversal and indel distance (D_{ri}) and average execution time (in seconds) of our heuristics, the MSOAR framework, and the ILP

O	$ \Sigma $	GA		RP		MSOAR		ILP	
		D_{ri}	Time	D_{ri}	Time	D_{ri}	Time	D_{ri}	Time
10	10	29.51	7.28	98.70	37.40	34.23	0.89	161.47	120.14
10	20	30.99	5.67	85.42	33.79	32.30	0.49	95.99	120.11
10	30	31.04	5.28	73.90	32.12	31.73	0.42	32.23	120.13
10	40	30.91	5.13	64.01	31.74	31.33	0.39	30.94	120.16
10	50	30.98	4.98	54.99	30.65	31.25	0.41	30.98	119.42
20	10	57.07	10.66	114.39	45.51	63.69	4.39	170.20	120.15
20	20	56.40	8.89	113.38	43.94	62.06	2.60	133.69	120.16
20	30	57.81	8.13	110.10	43.63	61.48	1.68	80.16	120.20
20	40	58.85	7.84	106.79	44.60	61.23	1.46	61.75	120.24
20	50	58.54	7.69	100.01	45.45	60.23	1.19	60.02	120.26
30	10	77.90	13.48	122.05	49.90	82.44	8.07	170.35	120.17
30	20	76.10	11.37	126.87	49.34	83.61	6.76	148.21	120.17
30	30	77.18	10.50	126.38	48.73	83.55	4.96	113.19	120.20
30	40	78.68	10.16	125.70	50.87	84.41	4.14	91.20	120.22
30	50	79.03	9.76	122.22	51.78	83.47	3.37	86.17	120.27
40	10	92.37	15.99	127.15	55.45	93.46	11.57	168.18	120.20
40	20	94.24	14.36	135.56	56.02	99.72	12.13	155.49	120.18
40	30	94.47	13.20	137.23	54.49	101.64	10.17	133.21	120.20
40	40	94.59	12.42	136.61	55.15	101.83	8.52	112.66	120.24
40	50	95.31	12.05	134.39	56.51	101.83	7.22	107.53	120.27
50	10	103.99	18.24	130.45	58.75	102.09	14.02	172.38	120.22
50	20	106.47	16.10	139.58	57.90	110.66	16.65	157.44	120.19
50	30	108.19	14.92	142.83	57.77	114.44	16.33	142.87	120.21
50	40	108.81	14.44	143.57	58.81	115.80	14.23	127.84	120.21
50	50	110.13	13.87	143.71	60.29	117.48	12.51	124.64	120.30

The shortest distances found are indicated in bold

value of O , we select 10 instances from the group of instances produced by reversals and run the GA heuristics 100 times for each one. In Table 2, we can see the average of the standard deviations for the resulting reversal distances, after applying the GA algorithm in the instances considering each O .

Table 3 shows the average reversal and indel distance, and the average execution time (in seconds) of the Genetic Algorithm (GA) heuristic with $r = 1000$, $p = 100$,

and $m = 0.2$ (according to the parameters defined in Sect. 4), the Random Packing (RP) heuristic with $r = 1000$, the MSOAR framework (MSOAR), and the ILP (ILP). These tests were performed with the first group of instances.

From Table 3, it is noticeable that the shortest average distances are achieved by the GA heuristic in most cases, except with $O = 50$ and $|\Sigma| = 10$, where MSOAR found the shortest distance. When $O = 10$ and $|\Sigma| = 50$, the ILP and the GA heuristics produce the same average distance. Besides, on average the RP heuristic produces larger distances than the MSOAR framework in all instances, and produces larger distances than the ILP in sets produced with alphabets of sizes 40 and 50, and with alphabet of size 30 and $O \neq 50$.

Regarding the running time, the MSOAR framework has the shortest running time, followed by the GA heuristic, which outperforms the RP heuristic. The ILP was executed until the cutoff time of 2 min, which was chosen because it is larger than the running time of all other algorithms.

Table 4 shows the average reversal, transposition, and indel distance, and the average execution time (in seconds) of the Genetic Algorithm (GA) heuristic with $r = 1000$, $p = 100$, and $m = 0.2$, the Random Packing (RP) heuristic with $r = 1000$, the MSOAR framework (MSOAR), and the ILP (ILP). These tests were performed with the second group of instances.

In Table 4, we see that the GA heuristic produces the shortest distances, on average, in almost all cases. The ILP produces the best results, on average, with $O = 10$ and $|\Sigma| \in \{40, 50\}$. Besides, on average the RP heuristic produces larger distances than the MSOAR framework in sets produced with $O \in \{10, 20, 30\}$, with $O = 40$ and $|\Sigma| \in \{10, 20, 30\}$, and with $O = 50$ and $|\Sigma| = 10$. Furthermore, on average, the RP heuristic produces larger distances than the ILP in sets produced with alphabets of sizes 30, 40, and 50, and with alphabet of size 20 and $O \in \{40, 50\}$.

Regarding the running time, the MSOAR framework still has the shortest running time, followed by the GA heuristic, which is faster than the RP heuristic. The ILP is executed until the cutoff time of 2 min.

6.2 Experiments with real biological data

To validate our algorithm with real biological data, we applied our Genetic Algorithm heuristic to calculate the reversal and indel distance between genomes from the Cyanorak 2.1 (Garczarek et al. 2020) system. We used a total of 97 genomes, being 51 *Synechococcus*, 43 *Prochlorococcus*, and 3 *Cyanobium*. The genome sizes (number of genes) range from 1834 to 4391. Considering the size of the alphabets, we have 1720, 2527.95, 3985 of minimum, average, and maximum. Considering the ratio between the number of replicas (genes with two or more copies) and the total number of genes in each genome, we have 0.35%, 1.93%, and 10.13% of minimum, average, and maximum ratios, respectively. Considering the gene with the highest occurrence in each genome, we have the values of 2, 7.8, and 92 for minimum, average, and maximum, respectively.

From these values, we see that the number of replicated genes compared to the size of the genomes is relatively small.

Table 4 Average reversal, transposition, and indel distance (D_{rti}) and average execution time (in seconds) of our heuristics, the MSOAR framework, and the ILP

O	$ \Sigma $	GA		RP		MSOAR		ILP	
		D_{rti}	Time	D_{rti}	Time	D_{rti}	Time	D_{rti}	Time
10	10	22.05	7.06	72.10	36.88	31.07	0.78	101.11	120.17
10	20	21.54	5.50	59.75	33.40	28.51	0.39	69.88	120.11
10	30	21.40	5.02	53.07	30.83	28.54	0.30	23.12	120.13
10	40	21.20	4.73	44.83	29.57	28.38	0.28	21.17	120.13
10	50	21.53	4.50	36.32	27.99	28.37	0.26	21.22	119.14
20	10	43.54	10.43	83.17	45.12	60.64	3.66	106.21	120.20
20	20	43.04	8.65	82.07	43.53	58.11	2.16	89.30	120.16
20	30	43.46	8.12	78.53	42.46	56.54	1.37	55.16	120.19
20	40	43.08	7.51	74.07	43.31	56.39	1.41	45.34	120.23
20	50	43.38	7.23	71.11	43.12	56.23	1.12	44.48	120.25
30	10	59.09	13.31	90.69	49.51	78.33	7.30	107.03	120.17
30	20	57.08	10.91	91.13	48.17	77.91	5.42	97.18	120.17
30	30	58.19	10.19	90.38	48.27	79.12	4.19	78.10	120.19
30	40	58.90	9.81	87.82	49.20	78.78	3.56	63.07	120.25
30	50	59.66	9.66	85.41	50.45	79.46	3.04	61.37	120.25
40	10	71.23	15.97	97.22	54.98	92.40	10.15	108.54	120.22
40	20	70.50	13.70	99.72	54.15	95.22	9.93	99.65	120.20
40	30	70.76	12.85	98.87	53.56	96.93	8.29	88.48	120.21
40	40	71.76	12.19	97.31	54.79	98.25	7.35	78.18	120.24
40	50	71.25	11.71	96.69	56.31	97.69	6.72	75.38	120.30
50	10	78.76	18.07	100.89	58.81	100.65	12.25	110.29	120.21
50	20	79.72	16.07	103.92	57.80	106.59	14.96	102.52	120.19
50	30	79.63	14.77	104.09	56.92	109.62	13.25	95.43	120.24
50	40	80.38	14.26	103.19	57.75	111.25	12.33	85.65	120.23
50	50	80.59	13.71	102.08	60.00	111.83	10.34	83.66	120.26

The shortest distances found are indicated in bold

In our tests, we applied the Genetic Algorithm heuristic (with $r = 100$, $p = 10$, and $m = 0.2$) to estimate the reversal and indels distances for each pair of genomes.

The tests took, on average, 56 s for each pair of genomes. Using the resulting distances, we constructed a phylogenetic tree⁴ with the Circular Order Reconstruction method (Makarenkov and Leclerc 1997), which is shown in Fig. 6. We can note that the genomes of organisms are, in general, grouped according to their clade. Our phylogenetic tree is considerably similar to the one presented by Garczarek et al. (2020). To compare the trees, we calculated the topological distance from Penny and Hendy (1985), which is defined as twice the number of branches defining different bipartitions of the leaf nodes. On our dataset with 97 genomes, the highest possible topological distance between two trees is equal to 192. The topological distance between the tree

⁴ Illustration created using *treeio* R package (Wang et al. 2020).

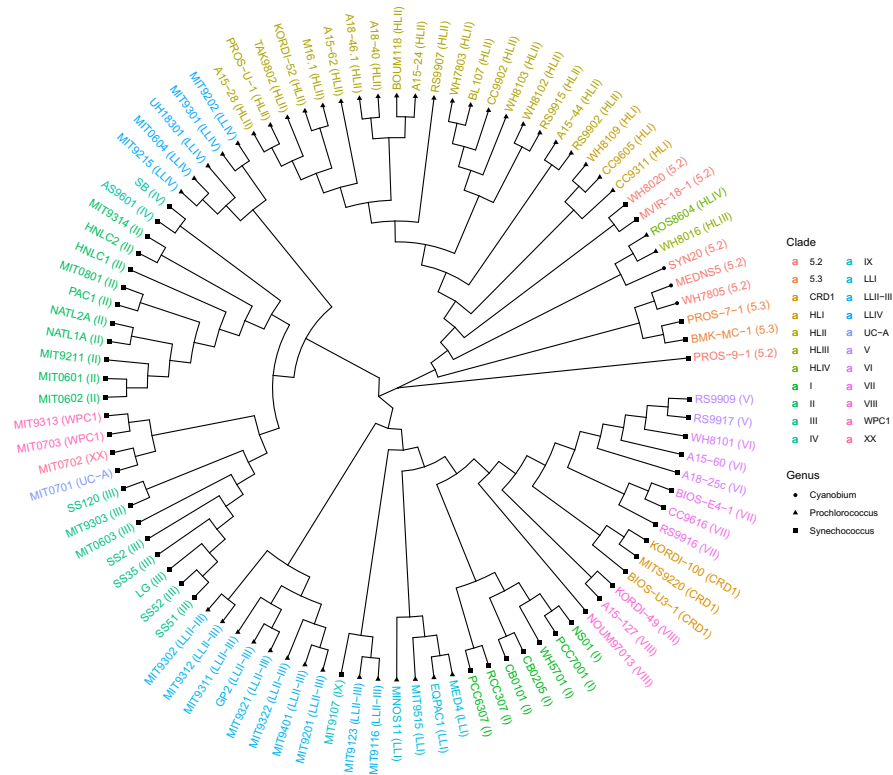


Fig. 6 Phylogenetic tree resulting from the reversal and indel distances obtained with the Genetic Algorithm heuristic

we obtained with our method and the tree from Garczarek et al. (2020) is 90, while the distance between a tree created with MSOAR and the tree from Garczarek et al. (2020) is 100.

7 Conclusion

In this work, we investigate the adjacency graph packing problem, which is an NP-hard problem when genomes have repeated genes. The goal of this problem is to find a cycle packing that maximizes a given gain function. We developed two heuristics, one that uses a random approach and one based on the Genetic Algorithm approach.

We implemented these heuristics and tested them on simulated and real datasets. We also compared both heuristics with the MSOAR framework. Although our heuristics took longer to run than MSOAR, our running times remained reasonable, and most importantly, they were able to find cycle packings with more cycles (on average) than the ones of MSOAR. Furthermore, using these packings to estimate the reversal and indel distance and the reversal, transposition, and indel distance allowed our heuristics to encounter shorter distances than the ones provided by the MSOAR framework.

Additionally, the GA heuristic found shorter distances, on average, than the PLI, limited in a running time above the one of the heuristics, in almost all cases.

As future works, our approach can be adapted to consider other characteristics of the genomes such as intergenic region information. It is also possible to develop other heuristics for the adjacency graph packing problem based on common metaheuristics such as GRASP, Tabu Search, and Simulated Annealing.

Acknowledgements This work was supported by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior—Brasil (CAPES)—Finance Code 001 and the São Paulo Research Foundation, FAPESP (Grants 2013/08293-7, 2015/11937-9, 2021/13824-8, and 2022/13555-0).

Declarations

Conflict of interest The authors have no relevant financial or non-financial interests to disclose.

References

- Alexandrino, A.O., Oliveira, A.R., Dias, U., Dias, Z.: Genome rearrangement distance with reversals, transpositions, and indels. *J. Comput. Biol.* **28**(3), 235–247 (2021)
- Alexandrino, A.O., Oliveira, A.R., Dias, U., Dias, Z.: Labeled cycle graph for transposition and indel distance. *J. Comput. Biol.* **29**(03), 243–256 (2022)
- Bafna, V., Pevzner, P.A.: Genome rearrangements and sorting by reversals. *SIAM J. Comput.* **25**(2), 272–289 (1996)
- Bafna, V., Pevzner, P.A.: Sorting by transpositions. *SIAM J. Discrete Math.* **11**(2), 224–240 (1998)
- Bergeron, A., Mixtacki, J., Stoye, J.: A unifying view of genome rearrangements. In: *International Workshop on Algorithms in Bioinformatics*, pp. 163–173 (2006). Springer, Berlin
- Bohnenkämper, L., Braga, M.D.V., Doerr, D., Stoye, J.: Computing the rearrangement distance of natural genomes. *J. Comput. Biol.* **28**(4), 410–431 (2021)
- Braga, M.D., Willing, E., Stoye, J.: Double cut and join with insertions and deletions. *J. Comput. Biol.* **18**(9), 1167–1184 (2011)
- Brito, K.L., Alexandrino, A.O., Oliveira, A.R., Dias, U., Dias, Z.: Reversals and transpositions distance with proportion restriction. *J. Bioinform. Comput. Biol.* **19**(04), 2150013 (2021)
- Chen, X., Zheng, J., Fu, Z., Nan, P., Zhong, Y., Lonardi, S., Jiang, T.: Assignment of orthologous genes via genome rearrangement. *IEEE/ACM Trans. Comput. Biol. Bioinf.* **2**(4), 302–315 (2005)
- Christie, D.A.: *Genome Rearrangement Problems*. Ph.D. thesis, Department of Computing Science, University of Glasgow (1998)
- Fu, Z., Chen, X., Vacic, V., Nan, P., Zhong, Y., Jiang, T.: MSOAR: a high-throughput ortholog assignment system based on genome rearrangement. *J. Comput. Biol.* **14**(9), 1160–1175 (2007)
- Garczarek, L., Guyet, U., Doré, H., Farrant, G.K., Hoebeke, M., Brillet-Guéguen, L., Bisch, A., Ferrieux, M., Siltanen, J., Corre, E., et al.: Cyanorak v2. 1: a scalable information system dedicated to the visualization and expert curation of marine and brackish picocyanobacteria genomes. *Nucleic Acids Res.* **49**, 1 (2020)
- Hannenhalli, S., Pevzner, P.A.: Transforming cabbage into turnip: polynomial algorithm for sorting signed permutations by reversals. *J. ACM* **46**(1), 1–27 (1999)
- Kahn, C., Raphael, B.: Analysis of segmental duplications via duplication distance. *Bioinformatics* **24**(16), 133–138 (2008)
- Makarenkov, V., Leclerc, B.: Circular orders of tree metrics, and their uses for the reconstruction and fitting of phylogenetic trees. In: *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*, pp. 183–208 (1997)
- Mitchell, M.: *Introduction to Genetic Algorithms*. Springer, Cambridge (2008)
- Oliveira, A.R., Brito, K.L., Alexandrino, A.O., Siqueira, G., Dias, U., Dias, Z.: Rearrangement distance problems: an updated survey. *ACM Comput. Surv.* **56**(8) (2024)

- Oliveira, A.R., Brito, K.L., Dias, U., Dias, Z.: On the complexity of sorting by reversals and transpositions problems. *J. Comput. Biol.* **26**, 1223–1229 (2019)
- Penny, D., Hendy, M.: The use of tree comparison metrics. *Syst. Zool.* **34**(1), 75–82 (1985)
- Pinheiro, P.O., Alexandrino, A.O., Oliveira, A.R., de Souza, C.C., Dias, Z.: Heuristics for breakpoint graph decomposition with applications in genome rearrangement problems. In: *Proceedings of the 13th Brazilian Symposium on Bioinformatics (BSB'2020)*, pp. 129–140 (2020)
- Radcliffe, A.J., Scott, A.D., Wilmer, E.L.: Reversals and transpositions over finite alphabets. *SIAM J. Discrete Math.* **19**(1), 224–244 (2005)
- Shao, M., Lin, Y., Moret, B.M.: An exact algorithm to compute the double-cut-and-join distance for genomes with duplicate genes. *J. Comput. Biol.* **22**(5), 425–435 (2015)
- Siqueira, G., Oliveira, A.R., Alexandrino, A.O., Dias, Z.: Heuristics for cycle packing of adjacency graphs for genomes with repeated genes. In: *Proceedings of the 14th Brazilian Symposium on Bioinformatics (BSB'2021)*, pp. 93–105 (2021)
- Walter, M.E.M.T., Dias, Z., Meidanis, J.: Reversal and transposition distance of linear chromosomes. In: *Proceedings of the 5th International Symposium on String Processing and Information Retrieval (SPIRE'1998)*, pp. 96–102. IEEE Computer Society, Los Alamitos, CA, USA (1998)
- Wang, L.-G., Lam, T.T.-Y., Xu, S., Dai, Z., Zhou, L., Feng, T., Guo, P., Dunn, C.W., Jones, B.R., Bradley, T., et al.: Treeio: an R package for phylogenetic tree input and output with richly annotated and associated data. *Mol. Biol. Evol.* **37**(2), 599–603 (2020)
- Willing, E., Stoye, J., Braga, M.: Computing the inversion-indel distance. *IEEE/ACM Trans. Comput. Biol. Bioinf.* **18**(6), 2314–2326 (2021)
- Zhai, S., Zhang, P., Zhu, D., Tong, W., Xu, Y., Lin, G.: An approximation algorithm for genome sorting by reversals to recover all adjacencies. *J. Comb. Optim.* **37**(4), 1170–1190 (2019)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.